Advance Java LAB

Rajasthan Technical University, Kota (Raj.)

Project Report

Project Name-

Book Stock Management System

Submitted By:

1. Khushal Vaishnav 24E1ARADM30P080

Under the supervision of

Er. Ram Babu Buri
Assistant Professor

Session: 2025-26

Department of Artificial Intelligence & Data Science

ARYA COLLEGE OF ENGINEERING & I.T.

SP-42, RIICO INDUSTRIAL AREA, KUKAS, JAIPUR

CERTIFICATE OF APPROVAL

Department of Artificial Intelligence & Data Science

ARYA COLLEGE OF ENGINEERING & I.T., JAIPUR

(Affiliated to Rajasthan Technical University, Kota)

The Project Report entitled "Book Stock Management System: Transactional Web-Based Library Inventory & Circulation Management Platform (AutoLibSys)" submitted by Khushal Vaishnav (24E1ARADM30P080) is hereby approved in partial fulfillment of the requirements for the award of the Degree of Bachelor of Technology (B.Tech) in Artificial Intelligence & Data Science during the academic session 2025–2026.

_____	_____
Er. Ram Babu Buri	Dr. Ashok Kumar Kajla
Project Mentor & Evaluator	Head of the Department
Assistant Professor	Professor, Dept. of AI & DS
Dept. of AI & DS, ACEIT	ACEIT, Jaipur

CERTIFICATE

Department of Artificial Intelligence & Data Science

ARYA COLLEGE OF ENGINEERING & I.T., JAIPUR

This is to certify that the work embodied in this Project Report entitled "Book Stock Management System: Transactional Web-Based Library Inventory & Circulation Management Platform (AutoLibSys)" has been carried out by the following student of Bachelor of Technology in Artificial Intelligence & Data Science, under my direct supervision and guidance at Arya College of Engineering & I.T., Jaipur, affiliated to Rajasthan Technical University, Kota, during the academic session 2025–2026:

1. Khushal Vaishnav (24E1ARADM30P080)

To the best of my knowledge, this report represents bona fide work carried out by the candidate and has not been submitted elsewhere for the award of any other degree or diploma.

Date: _____

Place: Jaipur

_____

Er. Ram Babu Buri

Assistant Professor & Project Guide

Department of Artificial Intelligence & Data Science

Arya College of Engineering & I.T., Jaipur

DECLARATION

I, Khushal Vaishnav (24E1ARADM30P080), student of Bachelor of Technology in Artificial Intelligence & Data Science at Arya College of Engineering & Information Technology, Jaipur, hereby declare that the project report entitled "Book Stock Management System: Transactional Web-Based Library Inventory & Circulation Management Platform (AutoLibSys)" submitted to Rajasthan Technical University, Kota, is an authentic record of my own research and developmental work.

I confirm that the implementation, source code, database architectures, and findings presented in this report were developed under the supervision of Er. Ram Babu Buri, Assistant Professor, Department of Artificial Intelligence & Data Science. I have adhered to all academic integrity guidelines and properly cited all secondary references and literature.

Student Name	Enrollment No.	Signature
Khushal Vaishnav (Lead)	24E1ARADM30P080	_____

Date: _____ | Place: Jaipur

ACKNOWLEDGEMENT

I express my profound gratitude and heartfelt appreciation to my project mentor, Er. Ram Babu Buri, Assistant Professor, Department of Artificial Intelligence & Data Science, Arya College of Engineering & I.T., for his inspirational intellectual direction, constructive criticism, and continuous encouragement throughout the architecture, database modeling, and implementation of the Book Stock Management System (AutoLibSys).

I extend my sincere thanks to Dr. Ashok Kumar Kajla, Professor & Head of the Department of Artificial Intelligence & Data Science, for providing cutting-edge computing lab facilities, institutional infrastructure, and an enriching academic environment.

I also convey my gratitude to all faculty members and technical staff of the Department of AI & DS for their valuable suggestions. Finally, I thank Rajasthan Technical University (RTU), Kota, for establishing an innovative academic curriculum that motivated me to build this enterprise library inventory and circulation solution.

Khushal Vaishnav (24E1ARADM30P080)
5th Semester, B.Tech in Artificial Intelligence & Data Science
Arya College of Engineering & I.T., Jaipur

ABSTRACT

Academic and institutional libraries have traditionally relied on manual accession registers and disconnected spreadsheets where book holdings, category classifications, and student borrowing records remain fragmented across physical ledgers. In day-to-day library administration, operators encounter severe operational overhead in verifying real-time book availability across Lending, Reference, and Online collections, preventing negative stock discrepancies during book edits, and auditing student issue and return transactions.

To resolve these systemic bottlenecks, the Book Stock Management System (branded in the user interface as AutoLibSys) has been engineered as a high-integrity, web-based library inventory and circulation management platform tailored for the Advance Java Laboratory (5th Semester B.Tech in Artificial Intelligence & Data Science) under Rajasthan Technical University (RTU), Kota. Built on a decoupled Multi-Tier Controller-View architectural design, the system harnesses Java SE 25, Jakarta EE 10 Servlets (Servlet 6.0), JavaServer Pages (JSP 3.1), Java Database Connectivity (JDBC), and MySQL 8.0 (book_stock_db), managed within Apache Tomcat 10.1.

Key capabilities include an interactive Book Stock Dashboard computing live copy KPIs across Lending, Reference, and Online classifications, dynamic catalog search and category filtering, structured book acquisition and editing with pessimistic row-level locking (SELECT ... FOR UPDATE), live category holding aggregation, and atomic ACID student book issue and return workflows. Maintained in the official GitHub repository (https://github.com/khushalvaishnav020140-cmd/BookStockManagement2), the platform achieves sub-150ms query execution times, 100% protection against SQL injection via parameterized PreparedStatements, deterministic try-with-resources connection management, and environment-variable credential isolation (DB_PASSWORD). This project establishes a reliable, transparent, and scalable paradigm for academic library stock administration.

TABLE OF CONTENTS

S. No.	Chapter / Section Title	Page No.
—	Cover Page	i
—	Certificate of Approval	ii
—	Certificate	iii
—	Declaration	iv
—	Acknowledgement	v
—	Abstract	vi
—	Table of Contents	vii
1	Chapter 1: Objective & Scope of the Project	1
2	Chapter 2: Theoretical Background	2
3	Chapter 3: Definition of Problem	3
4	Chapter 4: System Analysis & User Requirements	4
5	Chapter 5: System Planning (PERT & Gantt Analysis)	5
6	Chapter 6: Methodology Adopted & Hardware/Software Specifications	6
7	Chapter 7: Detailed Life Cycle of the Project (SDLC)	7
8	Chapter 8: ERD & Data Flow Diagrams (DFD)	9
9	Chapter 9: Process Algorithms, Flowcharts & Database Schema	10
10	Chapter 10: Input & Output Screen Design (UI Specification)	11
11	Chapter 11: Printout of the Code Sheet (Architectural Snippets)	12
12	Chapter 12: Testing & Quality Assurance	14
13	Chapter 13: User & Operational Manual	15
14	Chapter 14: Conclusions	15
15	Chapter 15: Future Enhancements	15
16	Chapter 16: References	16
17	Chapter 17: Bibliography	16

CHAPTER 1: OBJECTIVE & SCOPE OF THE PROJECT

1.1 Objective

The primary objective of the Book Stock Management System (AutoLibSys) is to architect, develop, and deploy a transaction-safe digital library inventory and circulation platform that transforms manual, paper-bound book stock registers into a modern, centralized web application. Specific engineering objectives include:

1. Centralized Book Stock Inventory: To digitize library holdings in a normalized MySQL 8.0 relational database (book_stock_db), tracking book titles, authors, categories, physical rack locations, call numbers, total copies, available copies, and book classifications (Lending, Reference, and Online).

2. Decoupled Multi-Tier Web Architecture: To implement a clean separation of concerns using Jakarta EE 10 Servlets (com.library.servlet), JavaServer Pages (src/main/webapp/*.jsp), and centralized JDBC connectivity (com.library.db.DBConnection) on Apache Tomcat 10.1.

3. Real-Time Dashboard & Multi-Criteria Catalog Search: To provide live SQL aggregation of copy counts across book classifications on the dashboard (/dashboard) alongside instant keyword search (title/author LIKE) and category filtering (/books).

4. Concurrency-Safe Stock Conservation: To enforce the stock conservation invariant (0 <= available_copies <= total_copies) using explicit JDBC transactions and pessimistic row-level locking (SELECT ... FOR UPDATE) during book edits (/editBook).

5. Atomic Student Circulation Management: To automate student book issuance (/issueBook) and return (/returnBook) workflows with LocalDate validation and automatic available copy decrement/increment.

6. Zero-Secret Credential Security: To eliminate hardcoded database passwords from source code by reading DB_USER and DB_PASSWORD strictly from operating system environment variables.

1.2 Scope

The operational scope of the Book Stock Management System is designed specifically for academic libraries and departmental book banks at institutions affiliated with Rajasthan Technical University (RTU), Kota, supporting the 5th Semester B.Tech in Artificial Intelligence & Data Science Advance Java Laboratory:

• Curriculum Alignment: Demonstrates core and advanced Java web capabilities including Jakarta Servlet lifecycle (doGet/doPost), JSP view dispatching, JDBC PreparedStatement parameterization, ACID transaction demarcation, and relational SQL joins.

• Complete Book Lifecycle Management: Full capabilities for library administrators to register new acquisitions, update bibliographic metadata, adjust total copy counts safely, and monitor live stock status badges.

• Category Organization & Analytics: Dynamic grouping of library holdings by academic category with live book count aggregation via relational LEFT JOIN queries.

• End-to-End Student Circulation Ledger: Systematic tracking of student borrowers (student_name, student_id), issue dates, due dates, return dates, and loan status (Issued vs. Returned).

• Cross-Platform Web Accessibility: Zero-client-installation access across desktop and mobile web browsers over standard HTTP/1.1 connections.

CHAPTER 2: THEORETICAL BACKGROUND

2.1 Jakarta Enterprise Edition (Jakarta EE 10 / Servlet 6.0 & JSP 3.1)

Jakarta Enterprise Edition 10 (Jakarta EE 10) provides an industry-standard, multithreaded runtime environment for server-side Java web applications. AutoLibSys leverages the Jakarta Servlet 6.0 and Jakarta Server Pages (JSP 3.1) specifications deployed on Apache Tomcat 10.1:

• Jakarta Servlets 6.0 (jakarta.servlet.http.HttpServlet): Server-side Java controller classes mapped via @WebServlet annotations that intercept HTTP GET and POST requests, validate form parameters, coordinate transactional database queries, and dispatch responses under a multithreaded container lifecycle.

• JavaServer Pages 3.1 (JSP): Server-side presentation templates that combine semantic HTML5 and responsive CSS3 with dynamic Java attribute binding to render real-time KPI cards, searchable inventory tables, and status banners.

2.2 Multi-Tier Controller-View Architectural Pattern

The application architecture decouples presentation, request control, and data persistence into three interconnected tiers:

• Model & Persistence Layer: Represents relational entities (books, categories, circulation) and centralized database connectivity (DBConnection.java, TestConnection.java), encapsulating SQL execution and ACID transaction rules.

• View Layer (src/main/webapp/): Renders seven interactive user interface screens (dashboard.jsp, books.jsp, addBook.jsp, editBook.jsp, categories.jsp, circulation.jsp, issueBook.jsp) styled with modern CSS Grid and Flexbox.

• Controller Layer (com.library.servlet): Eight dedicated servlets (DashboardServlet, BookStockServlet, AddBookServlet, EditBookServlet, CategoriesServlet, CirculationServlet, IssueBookServlet, ReturnBookServlet) that enforce validation, row locking, and RequestDispatcher forwarding or Post-Redirect-Get (PRG) routing.

2.3 Database Management System (MySQL 8.0)

Persistence is driven by MySQL 8.0 Community Server hosting the book_stock_db schema. MySQL InnoDB provides full ACID transaction compliance, foreign key referential integrity, row-level pessimistic locking (SELECT ... FOR UPDATE), and high-speed connectivity via the MySQL Connector/J 26.7.0 Type-4 JDBC driver (com.mysql.cj.jdbc.Driver).

2.4 Literature Review & Academic Foundations

The architectural formulation of AutoLibSys is grounded in recent peer-reviewed research on automated library management systems and transactional database concurrency:

• Web-Based Library Automation: Recent IEEE and IJRLS studies (Bagewadi et al., 2025; ICIPNP IEEE Xplore, 2023) confirm that Browser/Server (B/S) Java and MySQL architectures eliminate manual cataloging bottlenecks and provide instantaneous visibility into lending versus reference stock.

• Regional ILS Evaluation in Kota, Rajasthan: Comparative studies of Koha, SOUL 4.0, and e-Granthalaya across libraries in Kota, Rajasthan (Bhardwaj et al., 2026) highlight that while monolithic ILS suites suit large central libraries, departmental and college labs require lightweight, low-overhead web inventory portals.

• Pessimistic Concurrency Control in Inventory Databases: Database concurrency literature (Wu et al., VLDB) demonstrates that hot-spot inventory counters require serialized row-level locking (SELECT ... FOR UPDATE) to prevent lost updates and negative available stock anomalies during concurrent user transactions.

CHAPTER 3: DEFINITION OF PROBLEM

3.1 Problem Statement

In traditional academic and departmental libraries, book inventory and student borrowing records are maintained using physical paper registers or isolated desktop spreadsheets. When students request books or staff audit holdings, librarians must manually cross-reference acquisition logs against hand-written issue ledgers to determine how many copies of a title are physically available. This manual workflow leads to arithmetic drift, inability to distinguish Lending stock from Reference or Online holdings at a glance, slow catalog lookups, and stock corruption when book quantities are edited while copies are out on loan.

3.2 Existing System Problems

• Fragmented Stock Visibility: Manual registers cannot dynamically compute real-time available copies or segment holdings across Lending, Reference, and Online classifications.

• Stock Drift During Concurrent Operations: Updating book quantities or recording student loans without database transactions frequently results in available copy counts exceeding total copies or dropping below zero.

• Slow Multi-Criteria Search: Searching thousands of book entries by partial title, author name, or subject category in physical ledgers is time-consuming and error-prone.

• Unvalidated Circulation Records: Manual loan logs lack automated date validation (allowing due dates earlier than issue dates) and fail to prevent duplicate book return entries.

3.3 Proposed Solution: Book Stock Management System (AutoLibSys)

AutoLibSys introduces a unified, transaction-safe Java web platform that centralizes all library stock and circulation operations:

• Live Executive Dashboard: Displays real-time KPI cards for Lending Books, Reference Books, Online Books, and Total Book Copies alongside a complete master inventory table.

• Dynamic Catalog Search & Category Filtering: Enables instant filtering by partial title or author keyword (LIKE queries) combined with category dropdown selection.

• Pessimistic Row-Locked Stock Editing: Uses SELECT ... FOR UPDATE locking in EditBookServlet to ensure updated total copies never fall below currently issued copies.

• Atomic Issue & Return Engine: Synchronizes circulation records and books.available_copies inside single ACID transactions with automatic commit and rollback protection.

CHAPTER 4: SYSTEM ANALYSIS & USER REQUIREMENTS

4.1 Feasibility Study

A comprehensive feasibility analysis was conducted prior to implementation to evaluate technical, operational, and economic viability:

• Technical Feasibility: Java SE 25, Jakarta EE 10 Servlets, JSP 3.1, Apache Tomcat 10.1, and MySQL 8.0 are mature, industry-proven technologies with robust JDBC driver support, confirming full technical feasibility.

• Operational Feasibility: The AutoLibSys web interface requires only a standard web browser (Chrome, Firefox, Edge) and intuitive sidebar navigation, allowing librarians and evaluators to operate the system without specialized training.

• Economic Feasibility: Engineered entirely using open-source software (OpenJDK 25, Apache Tomcat 10.1, MySQL Community Server 8.0, Eclipse IDE), requiring zero licensing expenditure.

4.2 Functional Requirements

• FR-01: Dashboard KPI Aggregation: Route root requests to DashboardServlet (/dashboard) and display live copy totals for Lending Books, Reference Books, Online Books, and Total Book Copies.

• FR-02: Master Book Inventory Listing: Retrieve and display all book records joined with category names ordered by descending ID via BookStockServlet (/books).

• FR-03: Multi-Criteria Catalog Search: Filter books by partial title or author keyword (LIKE %keyword%) and category ID on the Book Stock screen.

• FR-04: New Book Acquisition: Validate and insert new book records (title, author, categoryId, location, callNumber, totalCopies, bookType) via AddBookServlet (/addBook) with available_copies = total_copies.

• FR-05: Row-Locked Book Editing: Lock the target book row (SELECT ... FOR UPDATE) in EditBookServlet (/editBook), verify totalCopies >= issuedCopies, and recalculate available_copies = totalCopies - issuedCopies.

• FR-06: Category Holding Aggregation: List all book categories with their associated book counts computed via LEFT JOIN in CategoriesServlet (/categories).

• FR-07: Atomic Book Issuance & Return: Record student book loans (/issueBook) and returns (/returnBook) while atomically decrementing or incrementing books.available_copies and displaying the ledger in CirculationServlet (/circulation).

4.3 Non-Functional Requirements

• Performance: Dashboard KPI aggregation, catalog search queries, and transactional updates complete in under 150ms (well within the 500ms benchmark).

• Reliability & Concurrency: 100% of stock-mutating operations execute inside explicit JDBC transactions with SELECT ... FOR UPDATE row locks and automatic rollback on failure.

• Security: Parameterized PreparedStatements prevent 100% of SQL injection attacks; database credentials are isolated in system environment variables (DB_USER, DB_PASSWORD).

• Maintainability & Portability: Clean package separation (com.library.db, com.library.servlet, webapp JSPs) packaged as a standard Jakarta EE 10 Web Application deployable across Windows and Linux.

CHAPTER 5: SYSTEM PLANNING (PERT & GANTT ANALYSIS)

5.1 Project Planning & Milestone Timeline

Development of the Book Stock Management System was executed across a structured 10-week software engineering lifecycle as documented in the project repository:

Phase / Week	Milestone Activity Description	Days	Dependencies
Week 1	Project Planning, Problem Identification & Objective Formulation	7	None
Week 2	Requirement Elicitation, Functional/Non-Functional Analysis & SRS Drafting	7	Week 1
Week 3	UML Modeling (Use Case, Class/Module & Activity Workflow Diagrams)	7	Week 2
Week 4	Database Schema Engineering (book_stock_db: categories, books, circulation) & ER Diagram	7	Week 3
Week 5	System Architecture Setup, Eclipse Web Project Structure & DBConnection JDBC Factory	7	Week 4

Phase / Week	Milestone Activity Description	Days	Dependencies
Week 6	Core Implementation: DashboardServlet, BookStockServlet, AddBookServlet & EditBookServlet	7	Week 5
Week 7	Categories & Circulation Modules (CategoriesServlet, CirculationServlet, Issue/Return) & Unit Testing	7	Week 6
Week 8	Frontend-Backend Integration, Pessimistic Locking (FOR UPDATE) & PRG Status Banners	7	Week 7
Week 9	End-to-End System Testing, Boundary Validation & Security Audit (PreparedStatement / Env Vars)	7	Week 8
Week 10	Apache Tomcat 10.1 Deployment Verification, UI Screenshots & Final Project Report	7	Week 9

5.2 Work Breakdown Structure (WBS)

• Subsystem 1 — Presentation Tier & AutoLibSys UI Shell (Lead: Khushal Vaishnav): Responsive sidebar layout, KPI cards, searchable inventory tables, and JSP views (dashboard.jsp, books.jsp, addBook.jsp, editBook.jsp, categories.jsp, circulation.jsp, issueBook.jsp).

• Subsystem 2 — Book Inventory & Search Controllers (Lead: Khushal Vaishnav): DashboardServlet and BookStockServlet implementing dynamic SQL aggregation, partial title/author LIKE filtering, and category filtering.

• Subsystem 3 — Book Acquisition & Row-Locked Modification (Lead: Khushal Vaishnav): AddBookServlet and EditBookServlet enforcing input validation, SELECT ... FOR UPDATE row locking, and issued-copy conservation.

• Subsystem 4 — Category & Transactional Circulation Engine (Lead: Khushal Vaishnav): CategoriesServlet, CirculationServlet, IssueBookServlet, and ReturnBookServlet managing student loans and atomic stock updates.

• Subsystem 5 — Database Architecture & JDBC Connectivity (Lead: Khushal Vaishnav): Normalized MySQL 8.0 book_stock_db schema, environment-secured DBConnection.java factory, and TestConnection.java verifier.

CHAPTER 6: METHODOLOGY ADOPTED & HARDWARE/SOFTWARE SPECIFICATIONS

6.1 Development Methodology

The project adopted an Iterative Agile-Waterfall Hybrid methodology. Requirements specification, UML modeling, and relational database normalization were established upfront with Waterfall rigor (Weeks 1–4), while servlet controllers, transactional locking logic, and responsive JSP interfaces were built, integrated, and tested through iterative weekly milestones tracked on GitHub (Weeks 5–10).

6.2 Hardware Requirements

Hardware Component	Minimum Technical Specification
Processor	Intel Core i3 / AMD Ryzen 3 (2.0 GHz Dual-Core or higher)
Primary Memory (RAM)	4 GB minimum (8 GB recommended for Eclipse + Tomcat + MySQL)
Storage (Disk Space)	2 GB free SSD / HDD space for JDK, Tomcat, and MySQL database
Display Monitor	1366 x 768 resolution minimum (Full HD 1920 x 1080 recommended)
Network Interface	Standard Ethernet / Wi-Fi interface (Localhost loopback & LAN access)

6.3 Software Requirements

Software Component	System Specification / Version
Operating System	Windows 10/11 (64-bit) or Linux (Ubuntu 22.04 LTS)
Programming Language	Java Development Kit (JDK / Java SE 25)
Web Technologies	Jakarta Servlets 6.0, JavaServer Pages (JSP 3.1), HTML5, CSS3

Software Component	System Specification / Version
Web Application Server	Apache Tomcat 10.1.x (Jakarta EE 10 Web Container)
Database Engine	MySQL Community Server 8.0 (InnoDB Storage Engine — book_stock_db)
Database Driver	MySQL Connector/J 26.7.0 (mysql-connector-j-26.7.0.jar)
Integrated Dev Environment	Eclipse IDE for Enterprise Java and Web Developers
Version Control	Git & GitHub Repository (khushalvaishnav020140-cmd/BookStockManagement2)

CHAPTER 7: DETAILED LIFE CYCLE OF THE PROJECT (SDLC)

7.1 Software Development Life Cycle Phases

1. Phase 1: Requirement Analysis & Elicitation: Analyzed academic library book stock challenges and RTU Advance Java Lab guidelines to establish 12 traceable functional requirements (FR-01 to FR-12) and quality benchmarks.

2. Phase 2: Architectural & Database Design: Modeled the 4-Tier Servlet/JSP/JDBC/MySQL architecture, Use Case and Class diagrams, and normalized the 3-table schema (categories, books, circulation) in book_stock_db.

3. Phase 3: Core Servlet & JSP Implementation: Authored the 8 Jakarta Servlets in com.library.servlet, the centralized DBConnection utility in com.library.db, and the 7 JSP views in src/main/webapp/.

4. Phase 4: Concurrency & Transaction Integration: Implemented explicit ACID transactions (setAutoCommit(false), commit(), rollback()) and row-level pessimistic locking (SELECT ... FOR UPDATE) across mutating servlets.

5. Phase 5: Verification & Quality Assurance: Executed unit, boundary, concurrency, and SQL injection test suites across all modules and verified connectivity via TestConnection.java.

6. Phase 6: Deployment & Documentation: Deployed the web archive on Apache Tomcat 10.1, captured live operational screenshots, and compiled the SRS, IEEE Research Paper, Presentation, and Final Project Report.

7.2 Module-Wise Architectural Decomposition & Engineering Responsibilities

To guarantee high modularity, maintainability, and transactional safety, the Book Stock Management System (AutoLibSys) is decomposed into 5 cohesive architectural modules engineered by Khushal Vaishnav (24E1ARADM30P080):

Module 1: Executive Dashboard & Real-Time KPI Analytics (Lead: Khushal Vaishnav — 24E1ARADM30P080)

Architected the AutoLibSys portal shell, fixed left navigation sidebar, and the executive dashboard (dashboard.jsp, DashboardServlet.java). Executes SQL GROUP BY and COALESCE(SUM(total_copies), 0) queries to compute live book copy totals segmented across Lending, Reference, and Online classifications alongside the complete master inventory table.

Module 2: Book Stock Catalog, Search & Category Filtering (Lead: Khushal Vaishnav — 24E1ARADM30P080)

Engineered the searchable book catalog and category organization subsystem (books.jsp, categories.jsp, BookStockServlet.java, CategoriesServlet.java). Supports dynamic multi-criteria filtering by partial title or author keyword (b.title LIKE ? OR b.author LIKE ?) combined with category dropdown selection, and computes per-category book counts using relational LEFT JOIN queries.

Module 3: Book Acquisition & Pessimistic-Locked Stock Modification (Lead: Khushal Vaishnav — 24E1ARADM30P080)

Developed the book registration and concurrency-safe stock editing workflows (addBook.jsp, editBook.jsp, AddBookServlet.java, EditBookServlet.java). Enforces strict server-side validation on required fields, positive copy counts, and valid book types. During book edits, acquires an exclusive row lock (SELECT total_copies, available_copies FROM books WHERE id = ? FOR UPDATE), verifies that new total copies are not less than currently issued copies, and recalculates available_copies = totalCopies - issuedCopies.

Module 4: Transactional Student Book Circulation Engine (Lead: Khushal Vaishnav — 24E1ARADM30P080)

Engineered the end-to-end student loan and return subsystem (circulation.jsp, issueBook.jsp, CirculationServlet.java, IssueBookServlet.java, ReturnBookServlet.java). Validates loan date ranges (! dueDate.isBefore(issueDate)) via java.time.LocalDate, locks target book and circulation rows with FOR UPDATE, inserts or updates loan status ('Issued' vs. 'Returned'), and atomically decrements or increments books.available_copies within explicit ACID transactions.

Module 5: Database Architecture & Environment-Secured JDBC Layer (Lead: Khushal Vaishnav — 24E1ARADM30P080)

Designed the normalized 3-table MySQL 8.0 schema (book_stock_db) and authored the centralized JDBC connection factory (DBConnection.java) and CLI diagnostic tool (TestConnection.java). Enforces runtime credential isolation via System.getenv("DB_USER") and System.getenv("DB_PASSWORD"), 100% parameterized PreparedStatement queries, and deterministic try-with-resources cleanup.

Module	Module Domain	Lead Architect	Core Architectural Artifacts	System Deliverable Impact
Module 1	Dashboard & KPI Analytics	Khushal Vaishnav 24E1ARADM30P080	dashboard.jsp, DashboardServlet, web.xml	Real-time Lending, Reference, Online & Total Copy KPIs
Module 2	Catalog Search & Categories	Khushal Vaishnav 24E1ARADM30P080	books.jsp, categories.jsp, BookStockServlet, CategoriesServlet	Dynamic title/author LIKE search & LEFT JOIN category counts
Module 3	Book Acquisition & Locked Edit	Khushal Vaishnav 24E1ARADM30P080	addBook.jsp, editBook.jsp, AddBookServlet, EditBookServlet	Validated book registration & FOR UPDATE row-locked edits
Module 4	Circulation Issue & Return	Khushal Vaishnav 24E1ARADM30P080	circulation.jsp, issueBook.jsp, CirculationServlet, Issue/ReturnServlets	Atomic student loan/return logging & live stock sync
Module 5	Database & JDBC Security	Khushal Vaishnav 24E1ARADM30P080	book_stock_db, DBConnection, TestConnection	3NF MySQL schema, env-var secret isolation & zero SQLi

CHAPTER 8: ERD & DATA FLOW DIAGRAMS (DFD)

8.1 Entity-Relationship Diagram (ERD) Specification

The relational schema of the Book Stock Management System (book_stock_db) comprises 3 normalized entities in Third Normal Form (3NF) enforcing primary key uniqueness, foreign key referential integrity, and stock conservation:

1. categories: Primary Key (id, INT AUTO_INCREMENT). Attributes: category_name (VARCHAR(100), NOT NULL). Represents master academic book classifications; 1-to-Many parent of books.

2. books: Primary Key (id, INT AUTO_INCREMENT). Attributes: title (VARCHAR(200), NOT NULL), author (VARCHAR(150), NOT NULL), category_id (INT, Foreign Key referencing categories.id), location (VARCHAR(100)), call_number (VARCHAR(50)), total_copies (INT, >= 1), available_copies (INT, >= 0), book_type (VARCHAR(20): 'Lending', 'Reference', 'Online').

3. circulation: Primary Key (id, INT AUTO_INCREMENT). Attributes: book_id (INT, Foreign Key referencing books.id), student_name (VARCHAR(150), NOT NULL), student_id

(VARCHAR(50), NOT NULL), issue_date (DATE, NOT NULL), due_date (DATE, NOT NULL), return_date (DATE, NULL), status (VARCHAR(20): 'Issued', 'Returned').

8.2 Data Flow Diagrams (DFD)

Level 0 DFD (Context Diagram):

The context diagram depicts external actors (Library Administrator / Operator and System / DB Administrator) interacting with the AutoLibSys boundary. The Library Administrator submits catalog search queries, new book registrations, stock edits, and student book issue/return requests, receiving real-time dashboard KPIs, filtered inventory tables, category counts, and circulation status confirmations.

Level 1 DFD:

Level 1 decomposes the system into five core functional processes: (1.0) Dashboard KPI Aggregation, (2.0) Multi-Criteria Catalog Search & Category Filtering, (3.0) Book Registration & Row-Locked Stock Modification, (4.0) Category Holding Aggregation, and (5.0) Transactional Book Issuance & Return Processing. Each process communicates with designated relational data stores in book_stock_db (D1: categories, D2: books, D3: circulation).

CHAPTER 9: PROCESS ALGORITHMS, FLOWCHARTS & DATABASE SCHEMA

9.1 Algorithm for Row-Locked Book Stock Modification (EditBookServlet)

The concurrency-safe book stock update sequence in EditBookServlet.doPost() operates as follows:

1. Step 1: Client submits HTTP POST request to /editBook with id, title, author, categoryId, location, callNumber, totalCopies, and bookType.

2. Step 2: Servlet sets UTF-8 encoding and validates that required strings are non-empty, totalCopies >= 1, and bookType is in {'Lending', 'Reference', 'Online'}.

3. Step 3: Obtain JDBC connection via DBConnection.getConnection() and begin an explicit transaction with con.setAutoCommit(false).

4. Step 4: Acquire an exclusive row lock via SELECT total_copies, available_copies FROM books WHERE id = ? FOR UPDATE.

5. Step 5: Compute issuedCopies = oldTotal - oldAvailable. If totalCopies < issuedCopies, invoke con.rollback() and return HTTP 400 Bad Request.

6. Step 6: Compute newAvailable = totalCopies - issuedCopies, execute parameterized UPDATE books statement, invoke con.commit(), and redirect to /books?updated=success.

9.2 Algorithm for Atomic Book Issuance & Return (IssueBookServlet & ReturnBookServlet)

1. Step 1 (Issue): Client submits POST /issueBook with bookId, studentName, studentId, issueDate, and dueDate.

2. Step 2 (Date Validation): Parse dates via LocalDate.parse() and verify ! dueDate.isBefore(issueDate); otherwise return HTTP 400.

3. Step 3 (Locked Issue): Disable auto-commit, lock book row via SELECT available_copies FROM books WHERE id = ? FOR UPDATE, and verify available_copies > 0.

4. Step 4 (Atomic Issue Commit): INSERT loan row into circulation (status = 'Issued'), UPDATE books SET available_copies = available_copies - 1, commit, and redirect to /circulation?message=issued.

5. Step 5 (Locked Return): On POST /returnBook, lock circulation row (FOR UPDATE), verify status == 'Issued', set return_date = CURRENT_DATE and status = 'Returned', increment books.available_copies by 1, commit, and redirect to /circulation?message=returned.

9.3 Database Data Dictionary

Detailed specifications of primary tables in the book_stock_db MySQL database:

Table Name	Column Name	Data Type	Constraint & Description
categories	id	INT AUTO_INCREMENT	Primary Key, Unique Category Identifier
categories	category_name	VARCHAR(100)	NOT NULL, Academic Book Category Name
books	id	INT AUTO_INCREMENT	Primary Key, Unique Book Identifier
books	title	VARCHAR(200)	NOT NULL, Full Book Title
books	author	VARCHAR(150)	NOT NULL, Primary Author Name
books	category_id	INT	Foreign Key referencing categories(id)
books	location	VARCHAR(100)	Physical Rack / Shelf Location Identifier
books	call_number	VARCHAR(50)	Library Classification Call Number
books	total_copies	INT	NOT NULL (>= 1), Total Inventory Copies
books	available_copies	INT	NOT NULL (0 <= available <= total), Available Stock

Table Name	Column Name	Data Type	Constraint & Description
books	book_type	VARCHAR(20)	NOT NULL ('Lending', 'Reference', 'Online')
circulation	id	INT AUTO_INCREMENT	Primary Key, Unique Circulation Transaction ID
circulation	book_id	INT	Foreign Key referencing books(id)
circulation	student_name	VARCHAR(150)	NOT NULL, Borrowing Student Full Name
circulation	student_id	VARCHAR(50)	NOT NULL, University Roll / Enrollment ID
circulation	issue_date	DATE	NOT NULL, Date Book Was Issued
circulation	due_date	DATE	NOT NULL (>= issue_date), Scheduled Return Due Date
circulation	return_date	DATE	NULLABLE, Actual Date Book Was Returned
circulation	status	VARCHAR(20)	NOT NULL ('Issued' or 'Returned')

CHAPTER 10: INPUT & OUTPUT SCREEN DESIGN (UI SPECIFICATION)

10.1 AutoLibSys Navigation Shell & Book Stock Dashboard (dashboard.jsp)

The application provides a responsive sidebar shell (#153b70 dark navy theme) linking Dashboard, Book Stock, Add New Book, Categories, and Circulation. The dashboard displays four color-accented KPI cards (Lending Books, Reference Books, Online Books, Total Book Copies), a '+ Add New Book' quick button, and a 9-column master inventory table with color-coded Book Type badges.

10.2 Book Stock Catalog & Multi-Criteria Search Screen (books.jsp)

Features an integrated filter bar with a text input for title/author keyword search, a dynamically populated category dropdown selector, and Search/Clear buttons. Renders a 10-column inventory table

highlighting positive available stock in green, flagging '0 (Out of Stock)' items in bold red, and providing per-row Edit buttons.

10.3 Add New Book & Edit Book Screens (addBook.jsp & editBook.jsp)

Provides structured two-column and single-column forms with HTML5 validation (required, maxlength, min='1') for capturing Book Title, Author Name, Category, Rack/Shelf Location, Call Number, Total Copies, and Book Type. The Edit Book screen pre-fills current database values and includes an informational banner explaining automatic available stock recalculation.

10.4 Book Categories Directory Screen (categories.jsp)

Renders a clean three-column summary table displaying Category ID, Category Name, and the live Number of Books associated with each category.

10.5 Book Circulation Ledger & Student Issue Screen (circulation.jsp & issueBook.jsp)

Displays a 9-column circulation table with amber 'Issued' and green 'Returned' status pills and an inline 'Return Book' button with confirmation prompt for active loans. The Issue Book screen populates a dropdown of books where available_copies > 0 and captures Student Name, Student ID, Issue Date, and Due Date.

CHAPTER 11: PRINTOUT OF THE CODE SHEET (ARCHITECTURAL SNIPPETS & TEMPLATES)

Note: In accordance with standard software engineering practices, institutional project guidelines, and code confidentiality principles, full production source files are maintained under version control within the official project GitHub repository (https://github.com/khushalvaishnav020140-cmd/BookStockManagement2). This chapter presents representative architectural code snippets from the repository illustrating the structural design and request-handling mechanics of each of the 5 project modules.

11.1 Module 1: Dashboard KPI Aggregation Controller Snippet (DashboardServlet.java)

Illustrates the controller retrieving aggregate copy counts by book_type and forwarding to dashboard.jsp:

```
// Module 1: Stock KPI Aggregation & Dispatch (DashboardServlet.java)
@WebServlet("/dashboard")
public class DashboardServlet extends HttpServlet {
    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse
resp)
            throws ServletException, IOException {
        int totalBooks = 0, lending = 0, reference = 0, online = 0;
        String sql = "SELECT book_type, COALESCE(SUM(total_copies), 0) "
                   + "AS cnt FROM books GROUP BY book_type";
        try (Connection con = DBConnection.getConnection();
```

```
             PreparedStatement ps = con.prepareStatement(sql);
             ResultSet rs = ps.executeQuery()) {
            while (rs.next()) {
                String type = rs.getString("book_type");
                int cnt = rs.getInt("cnt");
                if ("Lending".equalsIgnoreCase(type)) lending = cnt;
                else if ("Reference".equalsIgnoreCase(type)) reference =
cnt;
                else if ("Online".equalsIgnoreCase(type)) online = cnt;
            }
            totalBooks = lending + reference + online;
        } catch (SQLException e) {
            throw new ServletException("Error loading dashboard", e);
        }
        req.setAttribute("totalBooks", totalBooks);
        req.setAttribute("lendingBooks", lending);
        req.setAttribute("referenceBooks", reference);
        req.setAttribute("onlineBooks", online);
        req.getRequestDispatcher("/dashboard.jsp").forward(req, resp);
    }
}
```

11.2 Module 2: Dynamic Catalog Search & Category Filter Snippet (BookStockServlet.java)

Illustrates parameterized SQL query construction for keyword search and category filtering:

```
// Module 2: Inventory Search & Category Filter (BookStockServlet.java)
StringBuilder sql = new StringBuilder(
    "SELECT b.id, b.title, b.author, c.category_name, b.location, " +
    "b.call_number, b.total_copies, b.available_copies, b.book_type " +
    "FROM books b LEFT JOIN categories c ON b.category_id = c.id " +
    "WHERE 1=1");

if (search != null && !search.trim().isEmpty()) {
    sql.append(" AND (b.title LIKE ? OR b.author LIKE ?)");
}
if (categoryId != null) {
    sql.append(" AND b.category_id = ?");
}
sql.append(" ORDER BY b.id DESC");
```

11.3 Module 3: Pessimistic-Locked Book Edit Transaction Snippet (EditBookServlet.java)

Illustrates row-level locking (SELECT ... FOR UPDATE) and issued-copy conservation during stock edits:

```
// Module 3: Row-Locked Stock Update (EditBookServlet.java)
con.setAutoCommit(false);
String lockSql = "SELECT total_copies, available_copies "
               + "FROM books WHERE id = ? FOR UPDATE";
try (PreparedStatement ps = con.prepareStatement(lockSql)) {
    ps.setInt(1, id);
    try (ResultSet rs = ps.executeQuery()) {
```

```
            if (rs.next()) {
                oldTotal = rs.getInt("total_copies");
                oldAvailable = rs.getInt("available_copies");
            }
        }
    }
    int issuedCopies = oldTotal - oldAvailable;
    if (totalCopies < issuedCopies) {
        con.rollback();
        response.sendError(HttpServletResponse.SC_BAD_REQUEST,
            "Total copies cannot be less than issued copies.");
        return;
    }
    int newAvailable = totalCopies - issuedCopies;
    con.commit();
    response.sendRedirect(request.getContextPath() + "/books?
    updated=success");
```

11.4 Module 4: Atomic Book Issue & Return Circulation Snippet (IssueBookServlet.java)

Illustrates transactional book issuance with row locking and automatic available stock decrement:

```
// Module 4: Transactional Book Issue (IssueBookServlet.java)
con.setAutoCommit(false);
String checkSql = "SELECT available_copies FROM books "
                + "WHERE id = ? FOR UPDATE";
try (PreparedStatement checkPs = con.prepareStatement(checkSql)) {
    checkPs.setInt(1, bookId);
    try (ResultSet rs = checkPs.executeQuery()) {
        if (!rs.next() || rs.getInt("available_copies") <= 0) {
            con.rollback();
            response.sendError(400, "No copies available.");
            return;
        }
    }
}
String insSql = "INSERT INTO circulation(book_id, student_name, "
              + "student_id, issue_date, due_date, status) "
              + "VALUES (?, ?, ?, ?, ?, 'Issued')";
String updSql = "UPDATE books SET available_copies = "
              + "available_copies - 1 WHERE id = ?";
con.commit();
response.sendRedirect(request.getContextPath() + "/circulation?
message=issued");
```

11.5 Module 5: Environment-Secured JDBC Connection Factory Snippet (DBConnection.java)

Illustrates the centralized JDBC connection factory reading credentials from system environment variables:

```
// Module 5: JDBC Connection Factory (DBConnection.java)
package com.library.db;
import java.sql.Connection;
```

```
import java.sql.DriverManager;
import java.sql.SQLException;

public class DBConnection {
    private static final String URL =
        "jdbc:mysql://localhost:3306/book_stock_db";
    private static final String USER =
        System.getenv("DB_USER") != null ? System.getenv("DB_USER") :
"root";
    private static final String PASSWORD = System.getenv("DB_PASSWORD");

    public static Connection getConnection() throws SQLException {
        if (PASSWORD == null) {
            throw new SQLException("DB_PASSWORD env var is not set.");
        }
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            return DriverManager.getConnection(URL, USER, PASSWORD);
        } catch (ClassNotFoundException e) {
            throw new SQLException("MySQL JDBC Driver not found.", e);
        }
    }
}
```

CHAPTER 12: TESTING & QUALITY ASSURANCE

12.1 Testing Strategy

A multi-tiered testing methodology was executed encompassing Unit Testing (verifying standalone JDBC connectivity via TestConnection and servlet input validators), Integration Testing (verifying servlet-to-MySQL transactional flows across books, categories, and circulation), and Security & Concurrency Testing (verifying PreparedStatement SQL injection immunity and SELECT ... FOR UPDATE row locking).

12.2 Test Cases & Execution Matrix

TC No.	Test Case Description	Test Input	Expected Output	Status
TC-01	Load Book Stock Dashboard & KPIs	GET /dashboard	Renders Lending, Reference, Online & Total Copy KPIs	PASS
TC-02	Search books by title/author keyword	GET /books?search=Java	Displays matching rows via LIKE query	PASS
TC-03	Filter books by valid category ID	GET /books?category=1	Displays books belonging to Category 1	PASS

TC No.	Test Case Description	Test Input	Expected Output	Status
TC-04	Add new book with valid metadata	POST /addBook (totalCopies=5, Lending)	Inserts book with available_copies=5 ; redirects	PASS
TC-05	Add book with invalid/empty title or copies <= 0	POST /addBook (title='', totalCopies=0)	Returns HTTP 400 Bad Request; zero DB mutation	PASS
TC-06	Edit book total copies above issued count	POST /editBook (issued=2, newTotal=6)	Updates total=6, available=4 inside row lock	PASS
TC-07	Edit book total copies below issued count	POST /editBook (issued=2, newTotal=1)	Rolls back transaction; returns HTTP 400 error	PASS
TC-08	View categories and aggregated book counts	GET /categories	Renders category list with LEFT JOIN book_count	PASS
TC-09	Issue available book to student	POST /issueBook (valid dates, avail > 0)	Inserts 'Issued' row & decrements available_copies	PASS
TC-10	Issue book with due date before issue date	POST /issueBook (dueDate < issueDate)	Rejected by LocalDate check with HTTP 400	PASS
TC-11	Return active issued book	POST /returnBook (status='Issued')	Sets status='Returned', return_date=today, avail+1	PASS
TC-12	SQL Injection attempt in catalog search bar	search=' OR '1'='1	PreparedStatement binds literal text safely	PASS

12.3 Test Results Summary

All 12 critical functional, boundary, concurrency, and security test cases achieved 100% PASS status. Parameterized PreparedStatements eliminated SQL injection vulnerabilities, and pessimistic row-level locking (SELECT ... FOR UPDATE) prevented stock drift across all concurrent edit, issue, and return operations.

CHAPTER 13: USER & OPERATIONAL MANUAL

13.1 System Prerequisites

• Workstation: Any modern web browser (Google Chrome, Mozilla Firefox, Microsoft Edge, Apple Safari).

• Local Server Environment: Eclipse IDE for Enterprise Java, JDK 25, Apache Tomcat 10.1, MySQL Server 8.0.

• GitHub Source Repository: https://github.com/khushalvaishnav020140-cmd/BookStockManagement2

13.2 Installation & Local Setup Guide

1. Step 1: Clone the Git repository: git clone https://github.com/khushalvaishnav020140-cmd/BookStockManagement2.git

2. Step 2: Create the MySQL database book_stock_db and tables (categories, books, circulation) on localhost:3306.

3. Step 3: Set environment variables DB_USER=root and DB_PASSWORD=<your_mysql_password>.

4. Step 4: Import the project into Eclipse IDE as an 'Existing Dynamic Web Project' and verify mysql-connector-j-26.7.0.jar in WEB-INF/lib/.

5. Step 5: Configure Apache Tomcat 10.1 Server runtime in Eclipse, deploy the project, and access http://localhost:8080/BookStockManagement2/dashboard.

13.3 User Guide — Library Administrator & Operator Workflows

• Inventory & Search Workflow: Open Dashboard (/dashboard) to monitor Lending, Reference, Online, and Total Copy KPIs -> navigate to Book Stock (/books) to search by title/author or filter by category -> click '+ Add New Book' or 'Edit' to register or update holdings.

• Circulation Workflow: Navigate to Circulation (/circulation) -> click '+ Issue New Book' (/issueBook.jsp) -> select an available book, enter Student Name, Student ID, Issue Date, and Due Date -> submit to issue book -> click 'Return Book' on active loans when returned.

CHAPTER 14: CONCLUSIONS

The Book Stock Management System (AutoLibSys) successfully addresses the operational and data-integrity challenges inherent in traditional manual library registers. By providing a centralized, web-based Jakarta EE 10 application adhering to a clean multi-tier architecture, the platform eliminates manual ledger errors, standardizes category and rack tracking, and provides instantaneous visibility into library holdings.

The project demonstrated the robustness of Java SE 25, Jakarta Servlets 6.0, JSP 3.1, JDBC, and MySQL 8.0 deployed on Apache Tomcat 10.1. All functional requirements and non-functional benchmarks—including sub-150ms query response times, 100% PreparedStatement SQL injection immunity, environment-variable credential security, and pessimistic row-locked stock conservation—were successfully realized.

CHAPTER 15: FUTURE ENHANCEMENTS

1. Role-Based Access Control (RBAC) & Student Portal: Adding encrypted session authentication with separate privilege tiers for Librarians, Faculty, and Students to reserve books online.

2. Automated Overdue Fine Calculation & Email Alerts: Implementing scheduled background tasks to calculate overdue fines when current_date > due_date and dispatch automated email/SMS reminders.

3. Barcode & RFID Scanner Integration: Supporting optical barcode and RFID tag scanning for one-scan book issuance, return, and shelf stock verification.

4. One-Click PDF & Excel Audit Reports: Adding export utilities to download monthly category-wise stock audits and student circulation ledgers as formatted PDF and XLSX workbooks.

CHAPTER 16: REFERENCES

1. Eclipse Foundation. (2022). Jakarta EE 10 Platform & Jakarta Servlet 6.0 Specification. https://jakarta.ee/specifications/servlet/6.0/

2. The Apache Software Foundation. (2024). Apache Tomcat 10.1 Architecture and Configuration Guide. https://tomcat.apache.org/tomcat-10.1-doc/

3. Oracle Corporation. (2024). MySQL 8.0 Reference Manual and MySQL Connector/J JDBC Documentation. https://dev.mysql.com/doc/

4. Rajasthan Technical University, Kota. (2025). B.Tech Scheme and Syllabus for Advance Java Laboratory (Artificial Intelligence & Data Science).

5. IEEE Computer Society. (1998). IEEE Std 830-1998: IEEE Recommended Practice for Software Requirements Specifications.

6. Vaishnav, K. (2026). BookStockManagement2 Source Code Repository. GitHub. https://github.com/khushalvaishnav020140-cmd/BookStockManagement2

CHAPTER 17: BIBLIOGRAPHY

Books

1. Schildt, H. (2021). Java: The Complete Reference (12th Edition). McGraw-Hill Education.

2. Sierra, K., & Bates, B. (2008). Head First Servlets and JSP (2nd Edition). O'Reilly Media.

3. Murach, J., & Urban, M. (2014). Murach's Java Servlets and JSP (3rd Edition). Mike Murach & Associates.

Online Academic Documentation

1. Oracle Java SE Documentation: https://docs.oracle.com/en/java/javase/

2. Apache Tomcat 10.1 Documentation: https://tomcat.apache.org/tomcat-10.1-doc/

3. Jakarta EE 10 Specifications: https://jakarta.ee/specifications/

Journals & Research Papers (Literature Review Base)

1. IEEE ICIPNP. (2023). 'Design and Implementation of Web-Based Library Management System'. IEEE Xplore, ISBN: 979-8-3503-6347-0. https://ieeexplore.ieee.org/document/10965463/

2. Bagewadi, P., Mathapati, D., Gangur, B., Madiwalar, A., & Patil, S. S. (2025). 'Automating Libraries: The Role of Library Management Systems in Modern Library Operations'. International Journal of Research in Library Science (IJRLS), 11(2), 105-114.

3. Bhardwaj, P., Kumar, A., Soni, A., & Sharma, S. (2026). 'Performance Evaluation of Library Management Software Packages in Selected Libraries of Kota, Rajasthan: A Comparative Study of KOHA, SOUL, and E-Granthalaya'. IJEAB.

4. Moruf, H. A., & Abu, I. Z. (2024). 'Next Generation Platforms in Library Management Systems: A Review of Web-Based and Cloud-Based Software'. FUW Trends in Science & Technology Journal, 9(2), 86-93.

5. Sarma, G. K., Das, J. M., & Singh, A. K. (2024). 'Peeking at the Cataloguing Modules of Popular Open-Source ILS: Evergreen vs Koha'. Journal of Information and Knowledge, 61(1), 9-17.